2026

풀스택 개발자 양성과정

네트워크 프로그래밍 구현

1조 정승원, 김선길, 박형규, 박한결

목차

INDEX

01 | 자바 버전 확인 — java version 확인

02 | MySQL 데이터베이스 생성 — BookDB 생성 및 원격접속 계정 생성 + BookDB 권한 부여

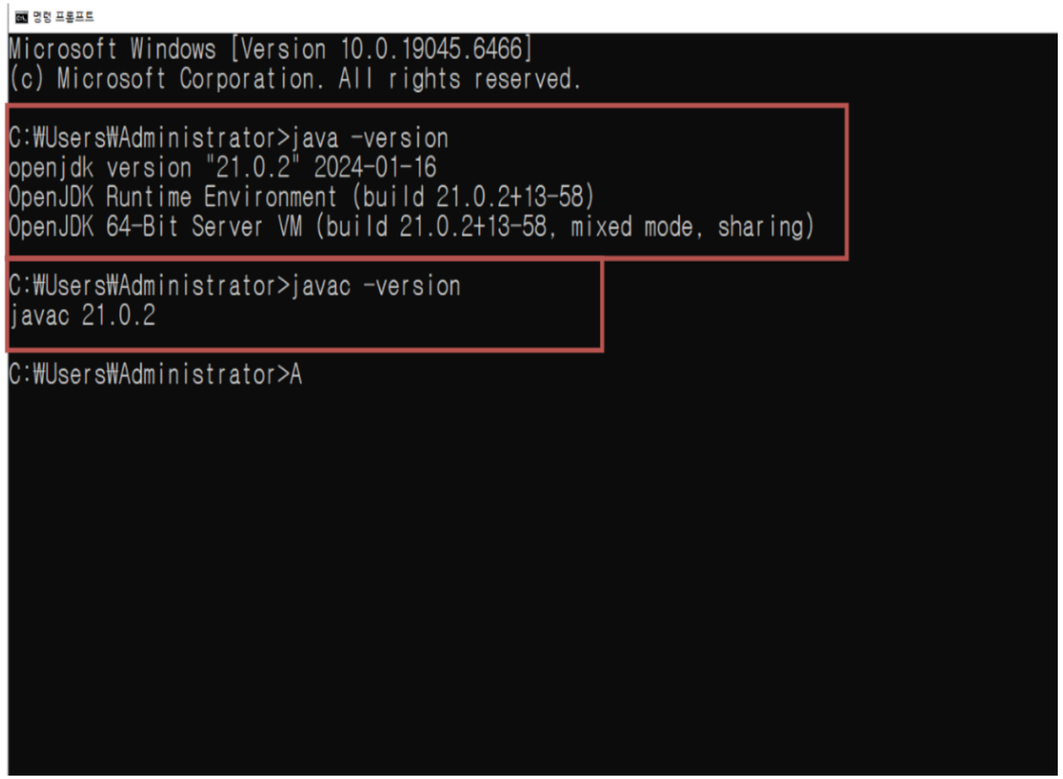
03 | JDBC Driver 등록 — Eclipse 빌드패스(.classpath)에 등록

04 | GitHub Reposity 페이지 — commit 로그 및 작업 페이지

05 | JDBC TODOS — 도서 정보(Book) 관리(CRUD) 기능 구현

06 | SOCKET TODOS — 도서관 운영 시스템에 직원 간 실시간 의사소통을 위한 다중 채팅 기능 구현

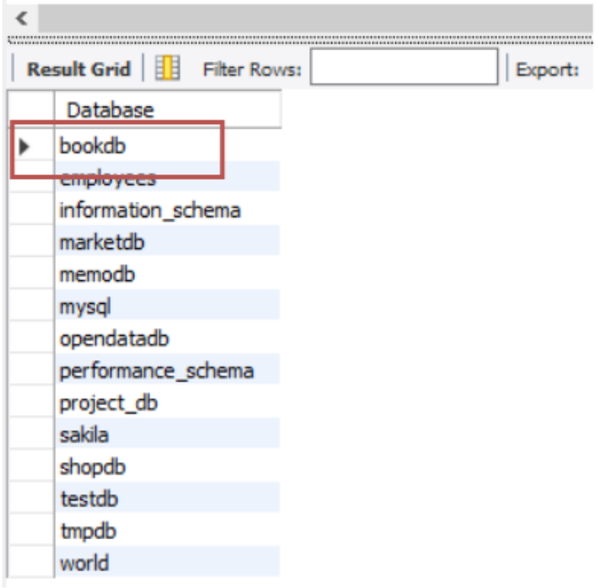
01 자바 버전 확인



1. java -version 확인
   version "21.0.2"로 확인 가능
2. javac -version 확인
   version "21.0.2"로 확인 가능

02 MySQL 데이터베이스 생성

```
1   show databases;
```

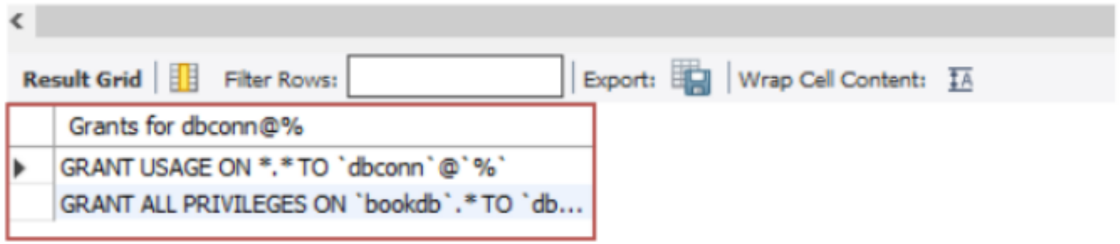


1. bookdb 생성

2. SHOW DATABASES; 를 통해 bookdb 생성 확인

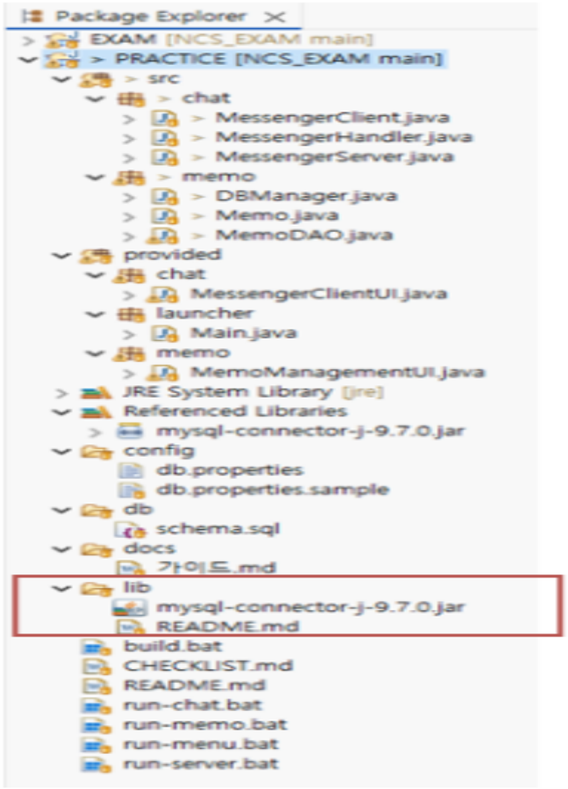
02 MySQL 데이터베이스 생성

```
1 • show grants for 'dbconn'@'%';
```



SHOW GRANTS FOR 'dbconn'@'%';
dbconn 사용자 원격 및 권한 확인

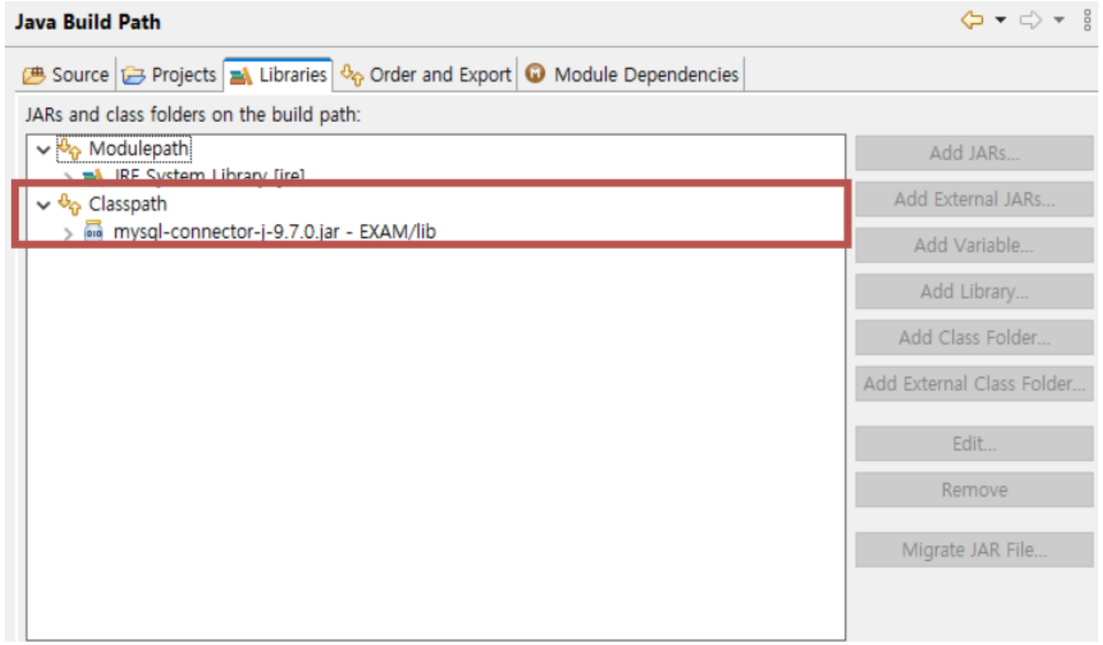
03 JDBC Driver 등록



1. mysql-connector-j-9.7.0 다운로드

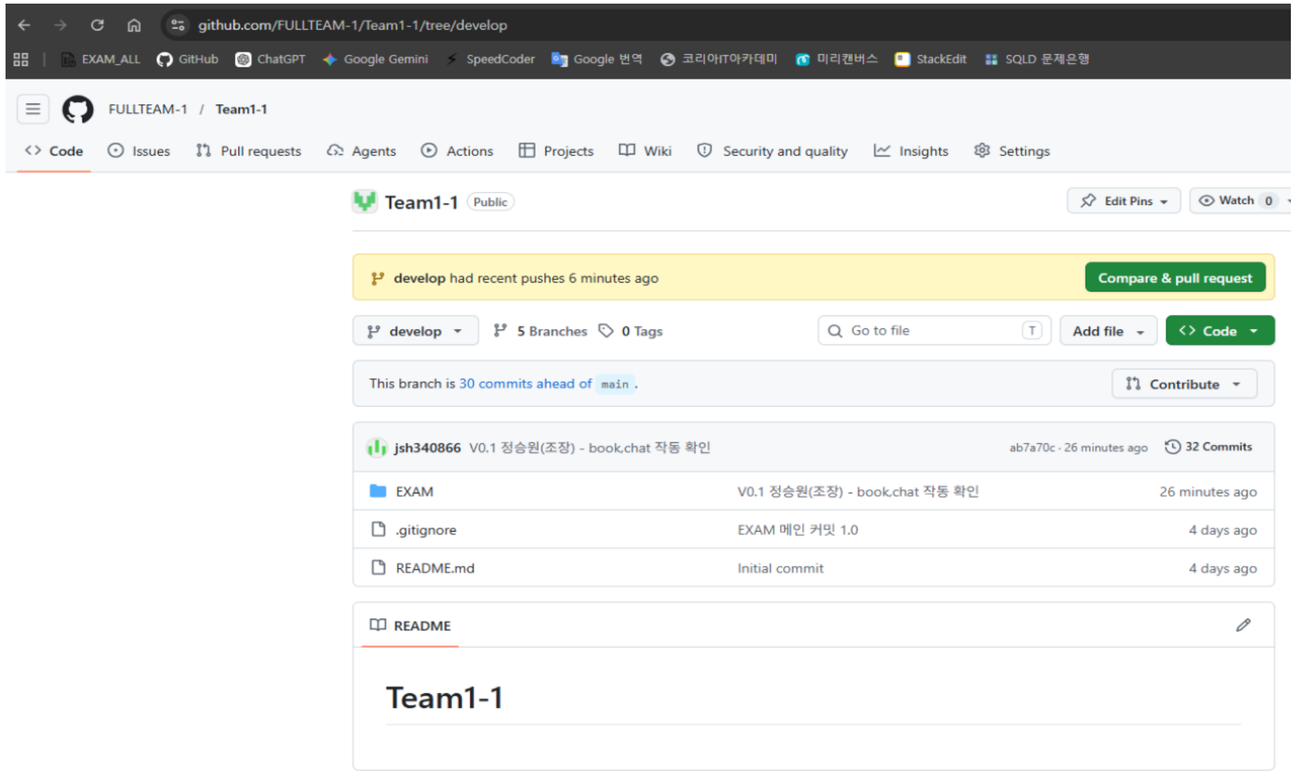
2. mysql-connector-j-9.7.0.jar
   lib폴더에 복사

03 | JDBC Driver 등록



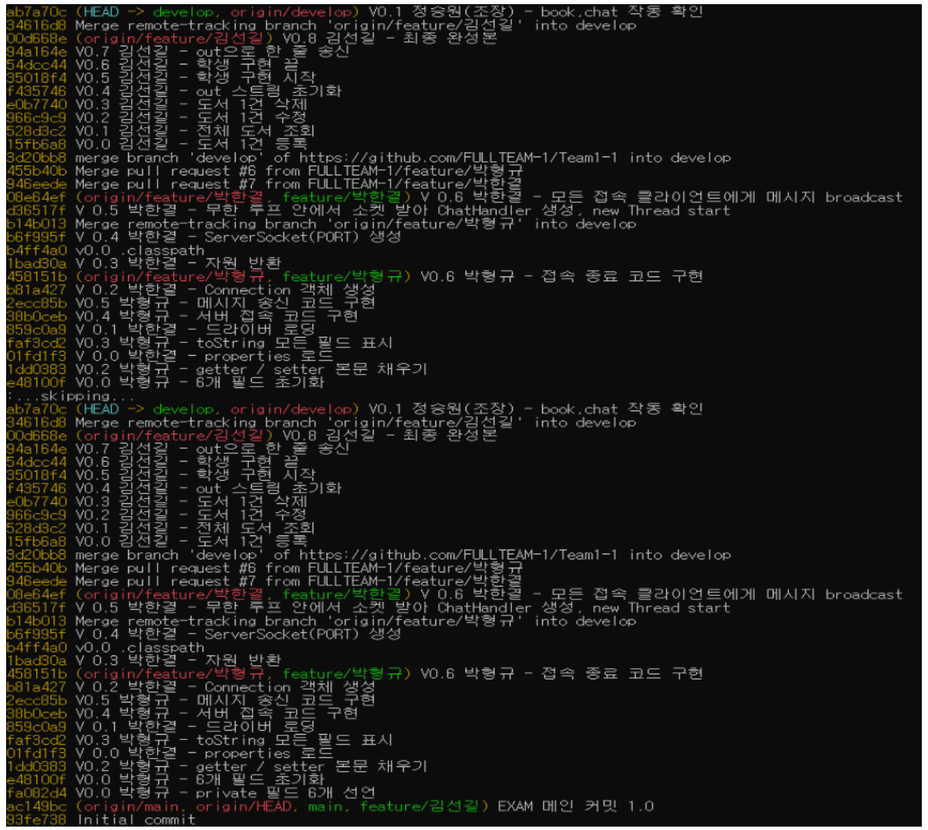
Classpath에 mysql-connector-j-9.7.0.jar 등록

04 GitHub Repository 페이지



작업 Repository 페이지

04 GitHub Repository 페이지



작업한 커밋 목록

05 JDBC TODOS (필드 선언)

```
public class Book {

    // TODO: private 필드 6개 선언
    private String BookCode;
    private int ClassificationId;
    private String author;
    private String name;
    private String publisher;
    private String isReserve;
```

1. Book 클래스 내부에 private로 6개의 필드 선언합니다.

2. 각각 String 타입인 BookCode, author, name, publisher, isReserve int 타입인 ClassificationId 필드로 선언하였습니다.

05 JDBC TODOS (Book 생성자 필드 초기화)

```java
public Book(String bookCode, Integer classificationId,
            String author, String name, String publisher, String isReserve) {
    // TODO: 6개 필드 초기화 (this.xxx = xxx)
    this.BookCode = bookCode;
    this.ClassificationId = classificationId;
    this.author = author;
    this.name = name;
    this.publisher = publisher;
    this.isReserve = isReserve;
}
```

1. Book 생성자 내부에 필드를 초기화하는 코드를 작성합니다.

2. Book 생성자의 6개의 매개변수를 받아 6개의 필드를 초기화하였습니다.

05 JDBC TODOS (getter / setter 채우기)

```
// ====== getter / setter — 학생 TODO: 본문 채우기 (시그니처는 변경 금지) ======
public String getBookCode() {
    return BookCode;
}

public void setBookCode(String bookCode) {
    BookCode = bookCode;
}

public int getClassificationId() {
    return ClassificationId;
}

public void setClassificationId(int classificationId) {
    ClassificationId = classificationId;
}

public String getAuthor() {
    return author;
}

public void setAuthor(String author) {
    this.author = author;
}
```

```
public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getPublisher() {
    return publisher;
}

public void setPublisher(String publisher) {
    this.publisher = publisher;
}

public String getIsReserve() {
    return isReserve;
}

public void setIsReserve(String isReserve) {
    this.isReserve = isReserve;
}
```

1. Book 클래스 내부에 모든 필드를 받아 getter / setter 메서드를 선언하였습니다.

05 JDBC TODOS (모든 필드 표시)

```java
/** 학생 TODO: 모든 필드 표시 (디버깅 용) */
@Override
public String toString() {
    return "Book[TODO=" + BookCode + ", Classification_Id=" + ClassificationId + ", author=" + author
            + ", name=" + name + ", publisher=" + publisher + ", isReserve=" + isReserve + "]";
}
```

1. toString()메서드를 활용하여 Book클래스의 반환되는 값을 출력할 수 있게 구현하였습니다.

2. return 반환값에 모든 필드를 작성하여 모든 필드를 출력할 수 있습니다.

05 BookDao 구현

```java
/**
 * 도서 1건 등록.
 *
 * <p>학생 TODO: try-with-resources 로 Connection / PreparedStatement 획득 →
 *    setString / setObject 로 6개 파라미터 바인딩 → executeUpdate() 반환.</p>
 */
public int insert(Book book) throws SQLException {
    // TODO: PreparedStatement 로 SQL_INSERT 실행
    try(Connection conn = DBUtil.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(SQL_INSERT)) {
        pstmt.setString(1, book.getBookCode());
        pstmt.setObject(2, book.getClassificationId());
        pstmt.setString(3, book.getAuthor());
        pstmt.setString(4, book.getName());
        pstmt.setString(5, book.getPublisher());
        pstmt.setString(6, book.getIsReserve());

        return pstmt.executeUpdate();
    }
}
```

V0.0) 도서 1건 등록

1. 도서 1건 등록 기능을 수행하기 위해 데이터베이스 연결 객체(Connection)와 SQL 실행 객체(PreparedStatement)를 생성하였습니다.

2. setString()과 setObject()를 사용하여 Book 객체에 저장된 도서코드, 분류번호, 저자, 도서명, 출판사, 대여 여부 데이터를 INSERT SQL문에 전달하였습니다.

3. executeUpdate()를 실행하여 도서 1건의 정보를 DB에 추가하고, 작업이 끝난 뒤 자원이 자동으로 해제되도록 구현하였습니다.

05 BookDao 구현

```java
/**
 * 전체 도서 조회.
 *
 * <p>학생 TODO: try-with-resources 로 Connection / PreparedStatement / ResultSet 획득 →
 *    rs.next() 루프 → Book 객체 채워서 List 에 추가.</p>
 *
 * <p>주의: <code>Classification_Id</code> 는 null 가능 컬럼.
 *    <code>rs.getInt(...)</code> 후 <code>rs.wasNull()</code> 로 null 체크 필요.</p>
 */
public List<Book> selectAll() throws SQLException {
    // TODO: PreparedStatement 로 SQL_SELECT_ALL 실행 → ResultSet 순회 → Book List 반환
    List<Book> list = new ArrayList<>();
    try(Connection conn = DBUtil.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(SQL_SELECT_ALL);
            ResultSet rs = pstmt.executeQuery()) {
        while(rs.next()) {
            String bookCode = rs.getString("Book_code");
            Integer classificationId = rs.getInt("classification_Id");
            if(rs.wasNull()) {
                classificationId = null;
            }
            String author = rs.getString("author");
            String name = rs.getString("name");
            String publisher = rs.getString("publisher");
            String isreserve = rs.getString("isreserve");

            Book book = new Book(
                    bookCode,
                    classificationId,
                    author,
                    name,
                    publisher,
                    isreserve
                    );
            list.add(book);
        }
    }
    return list;
}
```

V0.1) 전체 도서 조회

1. 전체 도서 조회 기능을 수행하기 위해 try-with-resources를 사용하여 Connection, PreparedStatement, ResultSet 객체를 생성하고 SELECT SQL문을 실행하였습니다.

2. ResultSet의 rs.next() 반복문을 통해 조회된 도서 데이터를 한 행씩 읽어온 뒤, 각 컬럼 값을 사용하여 Book 객체를 생성하고 List에 추가하였습니다.

3. Classification_Id 컬럼은 null 값이 가능하므로 rs.getInt() 호출 후 rs.wasNull()로 null 여부를 확인하여 처리하였으며, 최종적으로 전체 도서 목록을 반환하도록 구현하였습니다.

05 BookDao 구현

```java
/**
 * 도서 1건 수정 (Book_code 기준).
 *
 * <p>학생 TODO: SQL_UPDATE 실행. 마지막 파라미터(?)는 WHERE 의 Book_code.</p>
 */
public int update(Book book) throws SQLException {
    // TODO: PreparedStatement 로 SQL_UPDATE 실행
    try(Connection conn = DBUtil.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(SQL_UPDATE)) {
        pstmt.setObject(1, book.getClassificationId());
        pstmt.setString(2, book.getAuthor());
        pstmt.setString(3, book.getName());
        pstmt.setString(4, book.getPublisher());
        pstmt.setString(5, book.getIsReserve());
        pstmt.setString(6, book.getBookCode());
        return pstmt.executeUpdate();
    }
}
```

V0.2) 도서 1건 수정

1. 도서 1건 수정 기능을 수행하기 위해 try-with-resources를 사용하여 Connection과 PreparedStatement 객체를 생성하고 UPDATE SQL문을 실행하였습니다.

2. setObject()와 setString()를 사용하여 Book 객체의 분류번호, 저자, 도서명, 출판사, 대여 여부 데이터를 SQL문에 바인딩하였으며, 마지막 파라미터에는 수정 대상 도서를 찾기 위한 Book_code 값을 전달하였습니다.

3. executeUpdate()를 실행하여 해당 도서 정보를 수정하고, 작업 완료 후 자원이 자동으로 해제되도록 구현하였습니다.

05 BookDao구현

```
    /**
     * 도서 1건 삭제.
     *
     * <p>학생 TODO: SQL_DELETE 실행. ?에 bookCode 바인딩.</p>
     */
    public int delete(String bookCode) throws SQLException {
        // TODO: PreparedStatement 로 SQL_DELETE 실행
        try(Connection conn = DBUtil.getConnection();
                PreparedStatement pstmt = conn.prepareStatement(SQL_DELETE)) {
            pstmt.setString(1, bookCode);
            return pstmt.executeUpdate();
        }

    }
}
```

V0.3) 도서 1건 삭제

1. 도서 1건 삭제 기능을 수행하기 위해 try-with-resources를 사용하여 Connection과 PreparedStatement 객체를 생성하고 DELETE SQL문을 실행하였습니다.

2. setString()을 사용하여 SQL문의 값이 들어갈 위치(placeholder)에 삭제 대상 도서의 bookCode 값을 전달하였으며, 해당 도서코드와 일치하는 도서 정보를 기준으로 삭제가 이루어지도록 구현하였습니다.

3. executeUpdate()를 실행하여 도서 데이터를 삭제하고, 작업 완료 후 자원이 자동으로 해제되도록 구현하였습니다.

05 book/DBUtil 구현 (properties 로드)

```
    public static Connection getConnection() throws SQLException {
        // TODO 1) Properties 로 config/db.properties 로드
        Properties prop = new Properties();

        try (InputStream fin = new FileInputStream
("config/db.properties")) {
            prop.load(fin);
        } catch (IOException e) {
            throw new RuntimeException("DBManager's getConnection()
db.properties..."+e.getCause());
        };

        String driver   = prop.getProperty("db.driver");
        String url      = prop.getProperty("db.url");
        String user     = prop.getProperty("db.user");
        String password = prop.getProperty("db.password");
```

TODO 1) Properties 로 config/db.properties 로드

데이터베이스 접속 정보를 외부 파일 properties 로부터 읽어옵니다. Properties 객체와 FileInputStream을 활용해 외부 파일을 스트림으로 읽어들인 뒤, driver, url, user, password 정보를 각각의 문자열 변수에 할당합니다.

05 book/DBUtil 구현 (드라이버 클래스 메모리로 로드)

```
// TODO 2) Class.forName(driver) – JDBC 4단계 ①
try {
    Class.forName(driver);
} catch (ClassNotFoundException e) {
    throw new RuntimeException("DBManager's getConnection()
Class.forName..."+e.getCause());
}
```

TODO 2) Class.forName(driver) — JDBC 4단계 ①

Class.forName() 메서드를 통해 외부 파일에서 읽어온 MySQL 드라이버 클래스를 JVM 메모리에 로드합니다.

05 book/DBUtil 구현 (Connection 객체 생성)

```
        // TODO 3) return DriverManager.getConnection(url, user,
password); — JDBC 4단계 ②
        return DriverManager.getConnection(url, user, password);
    }
```

TODO 3) return DriverManager.getConnection(url, user, password); — JDBC 4단계 ②

앞에서 로드한 드라이버와 추출한 url, user, password 정보를 DriverManager.getConnection()에 전달하여 Connection 객체를 생성하고 반환합니다.

05 book/DBUtil 구현 (자원 닫기)

```
    public static void close(AutoCloseable... closeables) {
        // TODO: for-each + null 체크 + try { c.close(); } catch
(Exception ignored) {}
        for (AutoCloseable c : closeables) {
            if (c != null) {
                try {
                    c.close();
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        }

    }
```

TODO: for-each + null 체크 + try { c.close(); } catch (Exception ignored) {}

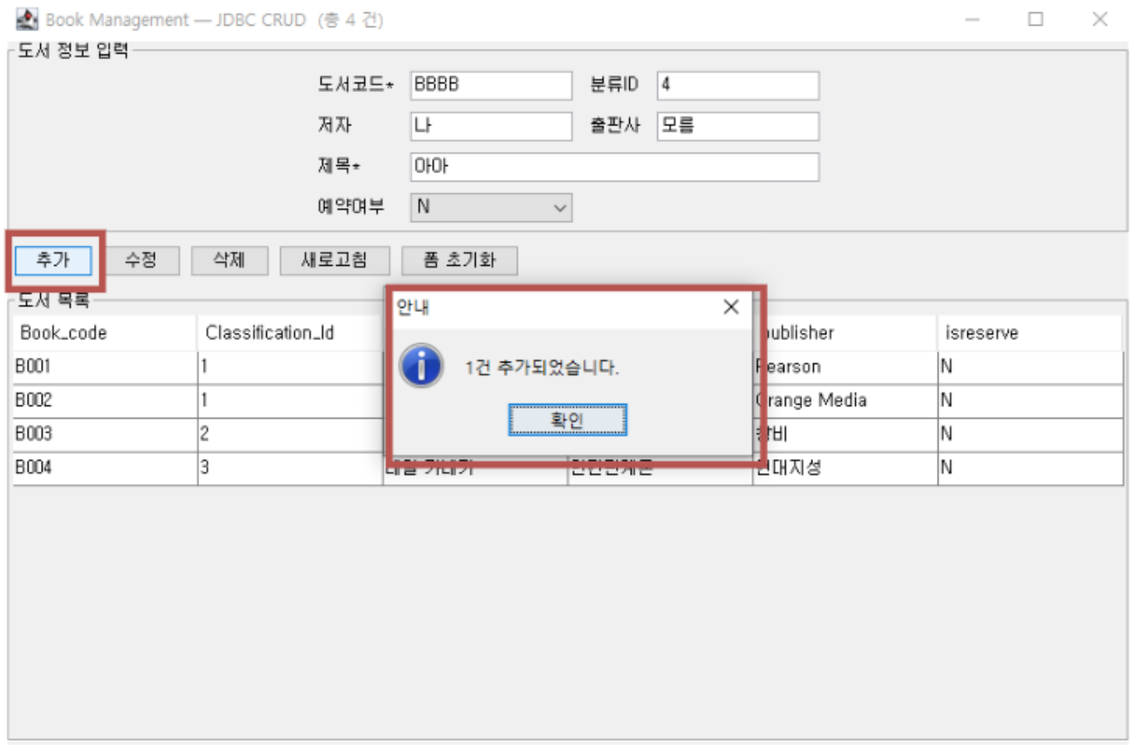
사용이 끝난 자원들을 닫습니다. 전달받은 자원들을 반복문으로 순회하며, null 여부를 확인한 후 close()를 호출합니다.자원을 닫는 도중 예외가 발생하더라도 다른 자원의 해제에 영향을 주지 않도록 내부에서 예외를 독립적으로 처리합니다.

05 JDBC CRUD 작동 결과 - Create



도서코드(BBB8), 분류ID(4), 저자(나), 출판사(모름), 제목(아아)을 원하는 값을 타입에 맞춰 작성

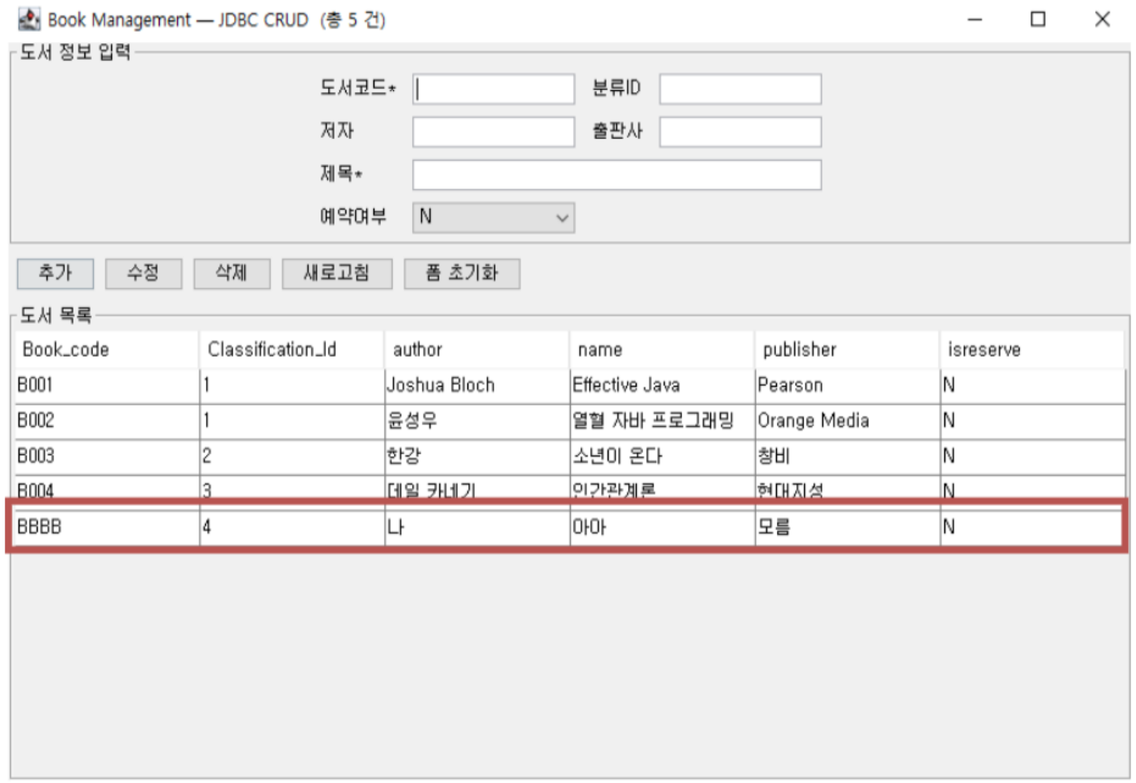
05 JDBC CRUD 작동 결과 - Create



1. 정보를 모두 입력한 후, 왼쪽 하단에 있는 [추가] 버튼을 눌러 DB에 저장합니다

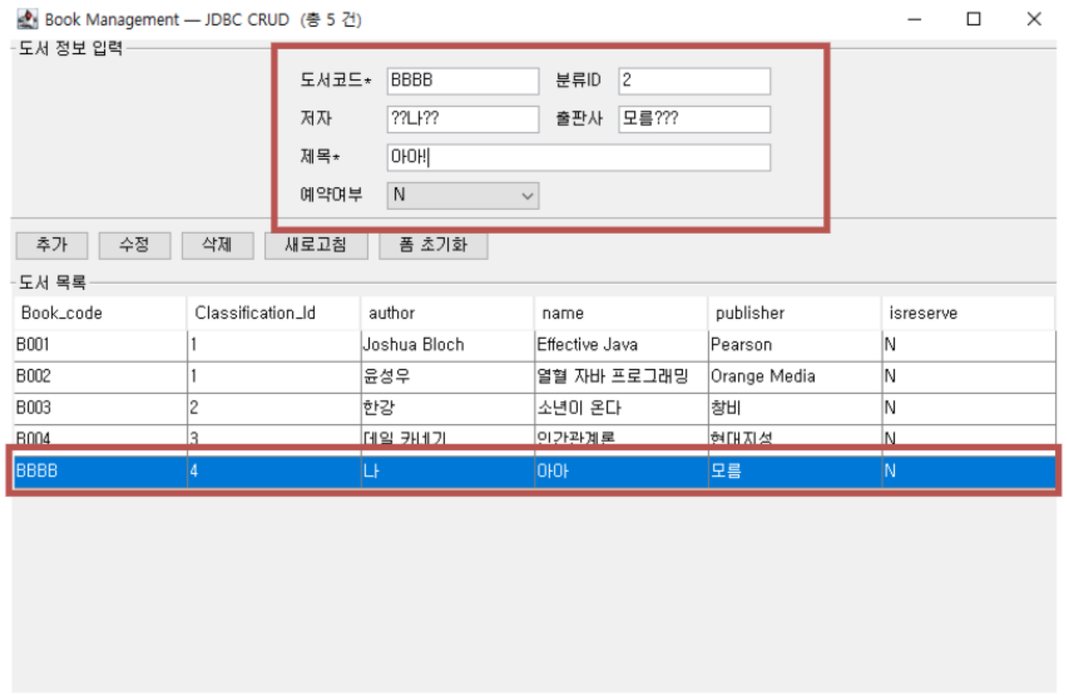
2. 화면에 "1건 추가되었습니다."라는 성공 안내 메시지 팝업창이 뜨면서, 저장이 잘 되었음을 확인할 수 있습니다

05 JDBC CRUD 작동 결과 - Create



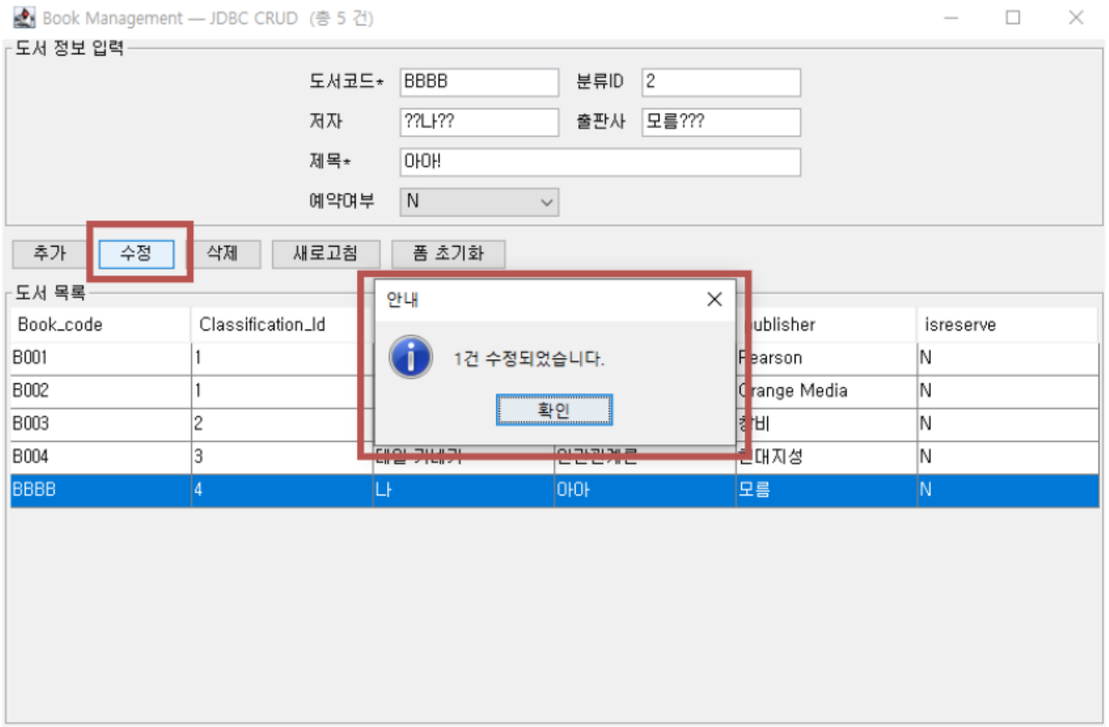
맨 아래 줄에 도서코드 BBBB번으로 데이터가 성공적으로 추가되어, 총 도서 건수가 4건에서 5건으로 늘어난 것을 확인할 수 있습니다.

05 JDBC CRUD 작동 결과 - Update



하단의 '도서 목록' 테이블에서 수정하고자 하는 도서 행(BBBB)을 클릭하여 선택한 뒤, 상단의 입력 창에서 변경할 내용을 작성합니다.

05 JDBC CRUD 작동 결과 - Update



1. 정보 수정을 마친 후 왼쪽 하단의 [수정] 버튼을 누르면, 데이터베이스(DB)의 해당 데이터가 업데이트됩니다.

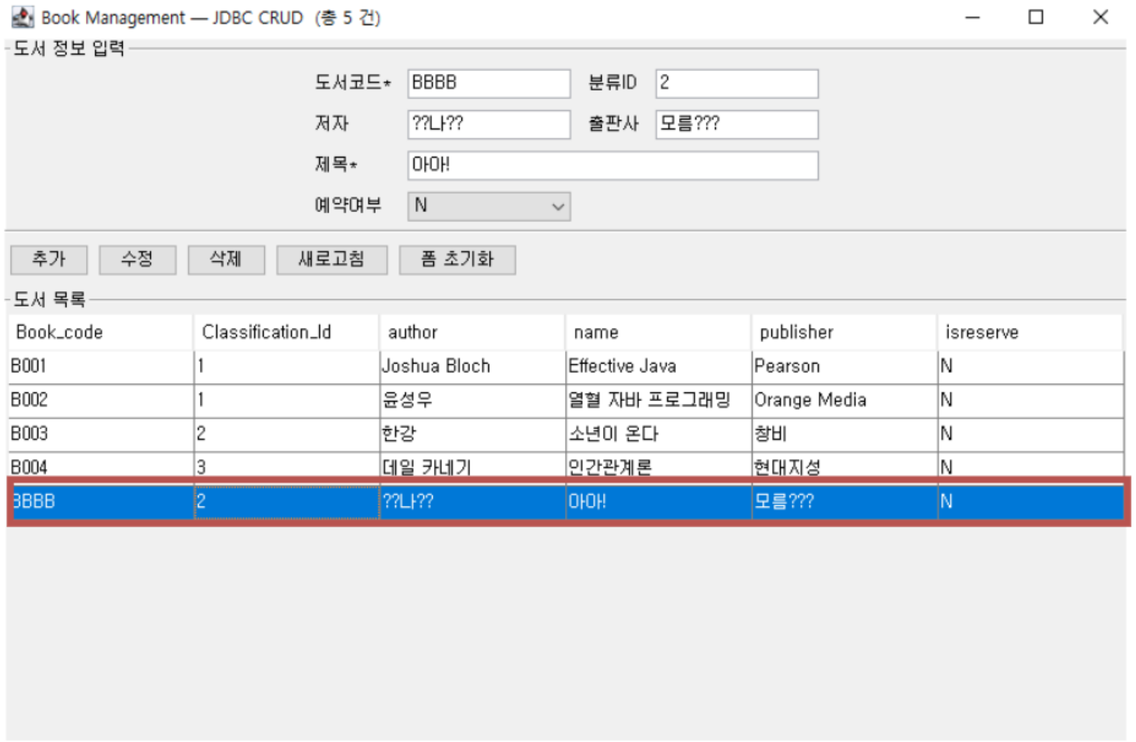
2. 버튼을 누르면 화면에 "1건 수정되었습니다."라는 성공 안내 메시지 팝업창이 표시되며 변경 사항이 정상적으로 반영되었음을 알려줍니다.

05 JDBC CRUD 작동 결과 - Update



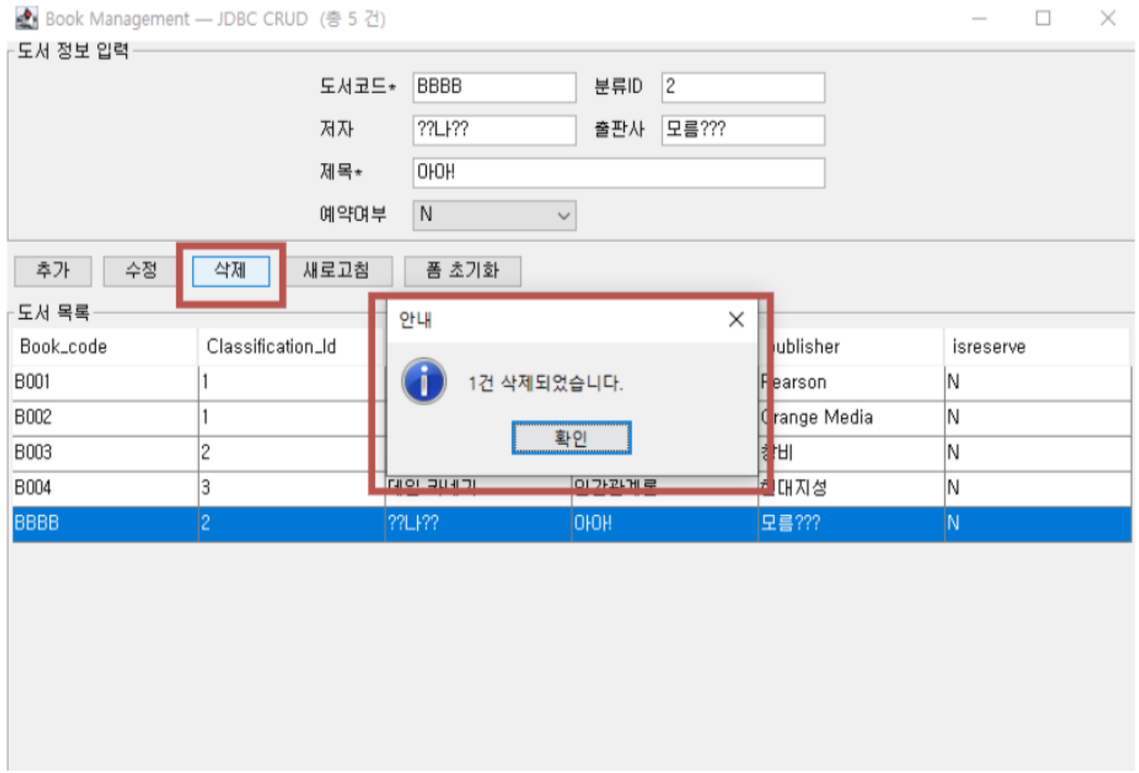
안내창의 [확인] 버튼을 누르면 하단의 '도서 목록' 테이블에 수정된 내용이 실시간으로 갱신되어 나타납니다.

05 JDBC CRUD 작동 결과 - Delete



하단의 '도서 목록' 테이블에서 데이터베이스(DB)에서 삭제하고자 하는 도서 행(BBBB)을 클릭하여 선택합니다.

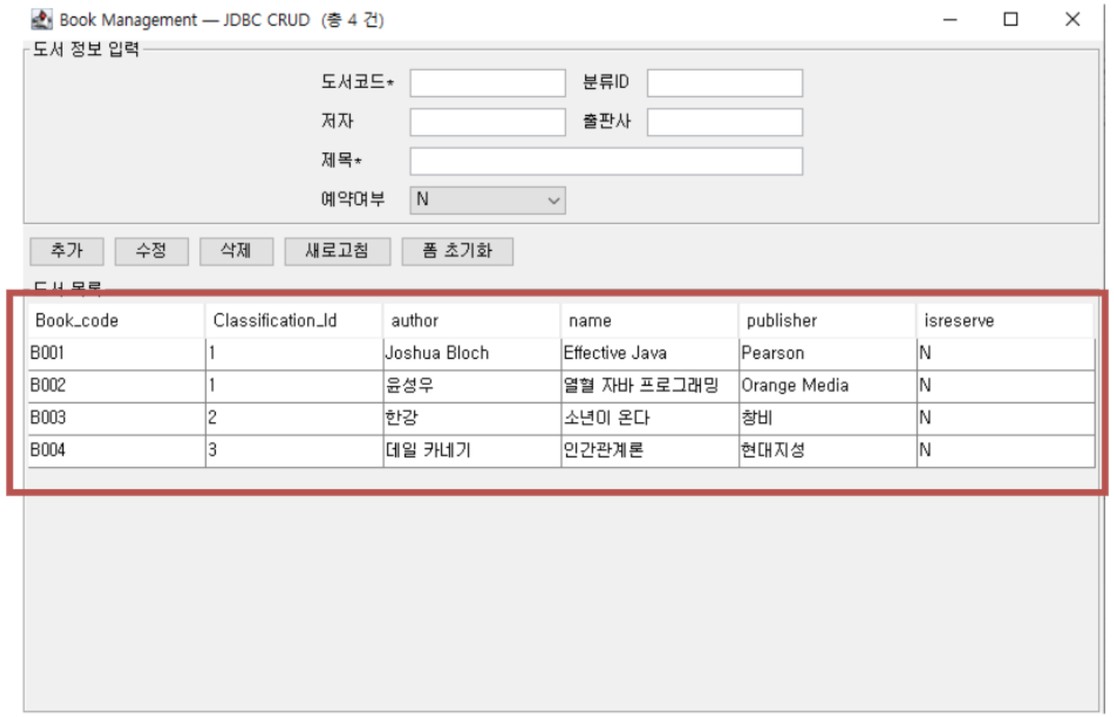
05 JDBC CRUD 작동 결과 - Delete



1. 삭제할 데이터를 확인한 후, 왼쪽 하단의 [삭제] 버튼을 누르면 데이터베이스(DB)에서 해당 레코드가 영구적으로 삭제됩니다.

2. 버튼 클릭 시 화면에 "1건 삭제되었습니다."라는 성공 안내 메시지 팝업창이 뜨며 데이터가 정상적으로 삭제되었음을 알려줍니다.

05 JDBC CRUD 작동 결과 - Delete



안내창의 [확인] 버튼을 누르면 상단의 입력 폼이 깨끗하게 초기화되고, 하단의 테이블 목록이 실시간으로 동기화됩니다.

06 SOCKET TODOS (서버 접속)

```java
/** 서버 접속 */
public void connect(String host, int port, String nickname) throws IOException {
    // TODO 1) socket = new Socket(host, port);
    socket = new Socket(host, port);
    // TODO 2) in / out 스트림 초기화 (UTF-8)
    InputStream tmpIn = socket.getInputStream();
    InputStreamReader in_reader = new InputStreamReader(tmpIn);
    in = new BufferedReader(in_reader);
    out = new PrintWriter(socket.getOutputStream());
    // TODO 3) out 으로 첫 줄에 닉네임 송신
    out.println(nickname);
    // TODO 4) running = true; → new Thread(this::readLoop).setDaemon(true) → start()
    running = true;
    readerThread = new Thread(this::readLoop);
    readerThread.setDaemon(true);
    readerThread.start();
}
```

Todo 1. socket = new Socket(host, port);을 써서 특정 주소의 서버에 연결 요청을 보냅니다.

Todo 2. 소켓에서 바이트 스트림을 가져와 한글이 깨지지 않고 한 줄씩 읽고 쓸 수 있는 문자열 입출력 스트림(in/out)을 초기화합니다.

Todo 3. 출력 스트림(out)을 통해 서버에 클라이언트의 닉네임을 첫 데이터로 전송하여 자신을 식별시킵니다.

Todo 4. 메인 흐름과 별개로 서버의 메시지를 실시간으로 상시 수신할 백그라운드 데몬 스레드를 생성하고 구동합니다.

06 SOCKET TODOS (메시지 송신)

```java
/** 메시지 송신 */
public void send(String message) {
    // TODO: out 이 null 이 아니면 println(message)
    if(out!=null) {
        out.println(message);
        out.flush();
    }
}
```

1. 서버와 연결이 안 되었거나 스트림이 생성되지 않은 상태에서 전송을 시도할 때 발생하는 NullPointerException 예외를 방지하기 위해 if문을 사용하였습니다.

2.매개변수로 받은 문자열(message) 뒤에 줄 바꿈 문자(\n)를 붙여서 출력 스트림의 버퍼에 기록합니다.

3.버퍼에 머물고 있는 메시지를 flush()메서드를 사용하여 네트워크를 통해 서버로 즉시 최종 방출합니다.

06 | SOCKET TODOS (접속 종료)

```java
/** 접속 종료 */
public void disconnect() {
    running = false;
    // TODO: socket → in → out 순서로 close (각각 null 체크 + try-catch 로 IOException 무시)
    //       ★ socket 을 먼저 닫아야 reader 스레드의 blocking read 가 깨어나 BufferedReader lock 이 풀림
    //       (Java 21 부터 BufferedReader 는 ReentrantLock 으로 보호 → 순서 잘못되면 EDT deadlock)
    try{if(socket!=null && !socket.isClosed()) socket.close();} catch (IOException e) {e.printStackTrace();}
    try{if(in!=null)in.close();}catch(IOException e) {e.printStackTrace();}
    if(out!=null)out.close();
}
```

1. running변수를 false로 바꾸어 스레드가 반복을 멈추고 종료되도록 유도합니다.

2.예외 처리문에서는 소켓이 연결되어 있다면 서버와의 물리적인 네트워크 연결을 즉시 끊어 통신을 완전히 종료하도록 합니다.

3.데이터를 보내던 출력 스트림 객체(PrintWriter)를 최종적으로 닫아(close()) 입출력 연결을 종료합니다.

06 ChatHandler 구현

```
 *   <li>생성자: Socket 의 InputStream / OutputStream 을 BufferedReader / PrintWriter 로 감싼다 (UTF-8 명시).</li>
 *   <li>run(): 첫 줄을 닉네임으로 받고, readLine() 반복하여 ChatServer.broadcast() 호출.</li>
 *   <li>send(): out.println(message) 한 줄.</li>
 *   <li>finally: ChatServer.remove(this) + in/out/socket close.</li>
 * </ol>
 *
 * <p>필요 import (추가): <code>java.io.InputStreamReader</code>, <code>java.io.OutputStreamWriter</code></p>
 */
public class ChatHandler implements Runnable {

    private final Socket socket;
    private BufferedReader in;
    private PrintWriter out;
    private String nickname = "guest";

    public ChatHandler(Socket socket) throws IOException {
        this.socket = socket;
        // TODO: in / out 스트림 초기화 (UTF-8 명시!)
        //   - in : BufferedReader + InputStreamReader(socket.getInputStream(), "UTF-8")
        //   - out: PrintWriter    + OutputStreamWriter(socket.getOutputStream(), "UTF-8"), autoFlush=true
        in = new BufferedReader(
                new InputStreamReader(
                        socket.getInputStream(),
                        "UTF-8"
                        )
                );
        out = new PrintWriter(
                new OutputStreamWriter(
                        socket.getOutputStream(),
                        "UTF-8"
                        ),
                true
                );

    }
```

V0.4) out 스트림 초기화(UTF-8) 명시

1. OutputStreamWriter와 PrintWriter를 사용하여 출력 스트림을 생성하였으며, "UTF-8" 을 명시하여 한글이 깨지지 않도록 구현하였습니다.

2. autoFlush=true 옵션을 설정하여 println() 실행 시 데이터가 자동으로 전송되도록 처리하였습니다.

06 ChatHandler 구현

```
    @Override
    public void run() {
        try {String name = in.readLine();
        if(name != null && !name.isBlank()) {
            nickname = name;
        }
        ChatServer.broadcast("[입장]" + nickname);

        String line;

        while((line = in.readLine()) !=null) {
            ChatServer.broadcast(
                    nickname +" : " + line);
        }
}
        catch(IOException e) {
            System.err.println(
                    "[ChatHandler]"
                    + nickname
                    + "접속 종료 : "
                    +e.getMessage());
```

V0.5) 학생 구현 시작

1. 클라이언트가 처음 입력한 값을 닉네임으로 저장한 뒤 입장 메시지를 전체 사용자에게 전송하도록 구현하였습니다.

2. readLine() 반복문을 통해 클라이언트의 채팅 메시지를 계속 입력받고, ChatServer.broadcast()를 사용하여 모든 사용자에게 채팅 내용을 전달하도록 구현하였습니다.

06 ChatHandler 구현

```
        // ====== 학생 구현 끝 ======================================
        } finally {
        // TODO 3) ChatServer.remove(this) 로 컬렉션에서 제거
            ChatServer.remove(this);
        // TODO 4) in / out / socket 자원 해제 (각각 null 체크 + try-catch)
            try {
                if(in != null)
                    in.close();
            }catch(IOException e) {
            }
            try {
                if(out!=null)
                    out.close();
            }catch(Exception e) {
            }
            try {
                if(socket!=null)
                    socket.close();
            }catch (IOException e) {
            }
        // TODO 5) 퇴장 broadcast
            ChatServer.broadcast("[퇴장]" + nickname);
    }
}
```

V0.6) 학생 구현 끝

1. 클라이언트 접속이 종료되면 현재 ChatHandler 객체를 서버의 클라이언트 목록(Collection)에서 제거하도록 구현하였습니다. 이를 통해 접속이 종료된 사용자가 서버 목록에 계속 남아있지 않도록 처리하였습니다.

2. 입력 스트림(in), 출력 스트림(out), 소켓(socket) 객체가 null이 아닌 경우 close()를 호출하여 사용이 끝난 자원을 해제하도록 구현하였습니다. 또한 try-catch를 사용하여 자원 해제 과정에서 발생할 수 있는 예외를 처리하도록 구성하였습니다.

3. 클라이언트 접속 종료 시 [퇴장] 메시지를 전체 사용자에게 전송하여 특정 사용자가 채팅방을 나갔음을 알리도록 구현하였습니다.

06 ChatHandler 구현

```
    /**
     * 이 핸들러가 담당하는 클라이언트에게 한 줄 전송.
     *
     * <p>학생 TODO: out 이 null 이 아니면 println(message) 호출.</p>
     */
    public void send(String message) {
        // TODO: out 으로 한 줄 송신
        if(out!=null)
            out.println(message);
    }

    public String getNickname() {
        return nickname;
    }
}
```

V0.7) out으로 한 줄 송신

1. send() 메서드는 현재 ChatHandler가 담당하는 클라이언트에게 메시지를 전송하기 위한 기능으로 구현하였습니다. 출력 스트림(out) 객체가 null이 아닌 경우 println()을 사용하여 전달받은 메시지를 클라이언트에게 한 줄 단위로 전송하도록 구성하였습니다.

2. getNickname() 메서드는 현재 클라이언트의 닉네임 값을 반환하기 위한 getter 메서드로 구현하였습니다.

06 chat/ChatServer 구현 (서버소켓 생성)

```java
public class ChatServer {

    public static final int PORT = 9999;

    /** 접속한 모든 클라이언트 핸들러 (스레드 안전 컬렉션) */
    public static final Set<ChatHandler> clients =
            Collections.synchronizedSet(new HashSet<>());

    public static void main(String[] args) {
        ServerSocket server = null;
        try {
            // ====== 학생 구현 시작 ======================================
            // TODO 1) ServerSocket(PORT) 생성 + 시작 로그 출력
            server = new ServerSocket(PORT);
            System.out.println("[ChatServer] 서버 시작 = PORT: " + PORT);
```

TODO 1) ServerSocket(PORT) 생성 + 시작 로그 출력

지정된 포트(9999)를 열고 클라이언트의 접속을 기다립니다. ServerSocket 객체를 생성하면서 포트 번호를 인자로 넘겨주어 해당 포트를 선점(바인딩)한 상태로 대기합니다.
서버가 정상적으로 구동되었음을 확인할 수 있도록 콘솔에 시작 로그를 출력합니다.

06 chat/ChatServer 구현 (client 소켓 받기)

```
            while (true) {
                // TODO 2) 무한 루프 안에서 accept() 로 Socket 받기
                Socket client = server.accept();

                // TODO 3) 받은 Socket 으로 ChatHandler 생성 → clients 에 add → 새
Thread 로 start
                ChatHandler handler = new ChatHandler(client);
                clients.add(handler);
                new Thread(handler).start();
            }
```

TODO 2) 무한 루프 안에서 accept() 로 Socket 받기
TODO 3) ChatHandler 생성 후 새 Thread 로 start

무한루프 안에서 소켓을 받아 접속자를 계속 받습니다.
server.accept()는 클라이언트가 접속할 때까지 대기(블로킹)하다 접속하면 통신 소켓을 반환합니다. 이를 무한 루프 안에서 실행해 접속자를 계속 받습니다.
소켓을 받으면 해당 소캣의 ChatHandler 객체를 생성 후 clients에 추가해 보관합니다. 그리고 새 Thread를 생성해 실행시켜 동시처리할 수 있게 합니다.

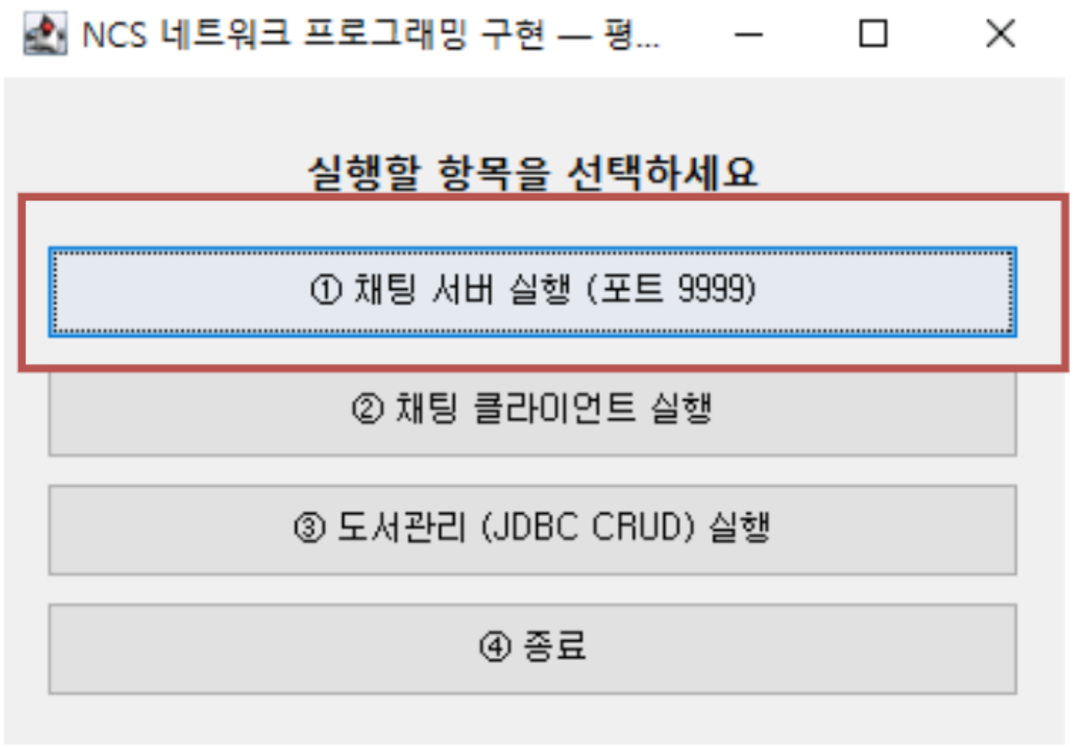
06 chat/ChatServer 구현 (handler.send(message) 호출)

```
public static void broadcast(String message) {
    // TODO: 동기화 블록 안에서 clients 순회 → 각 handler.send(message) 호출
    synchronized (clients) {
        for (ChatHandler h : clients) {
            h.send(message);
        }
    }
}
```

TODO: 동기화 블록 안에서 clients 순회 → 각 handler.send(message) 호출

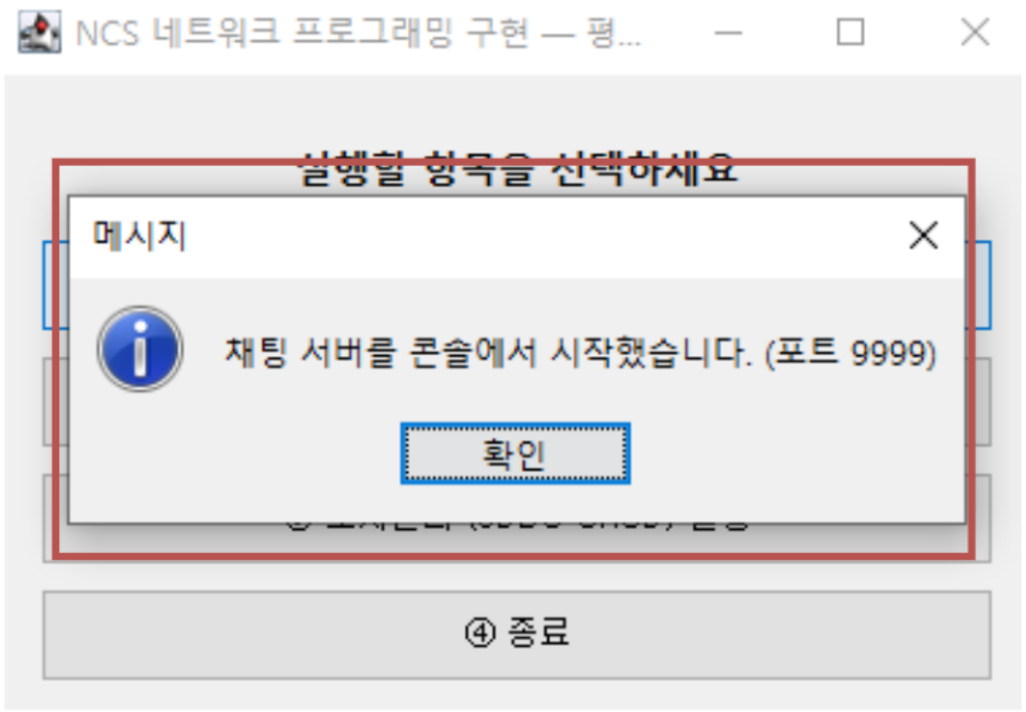
현재 채팅방에 접속 중인 모든 사용자에게 메시지를 동시에 전송(브로드캐스트)합니다. 먼저 데이터가 꼬이지 않도록 synchronized (clients) 동기화 블록을 설정하여 안전하게 보호합니다. 이후 내부에서 현재 접속 중인 모든 핸들러 객체를 반복문(for-each)으로 순회하며 send(message) 메서드를 호출해 메시지를 전송합니다.

06 | SOCKET TODOS - 작동 결과



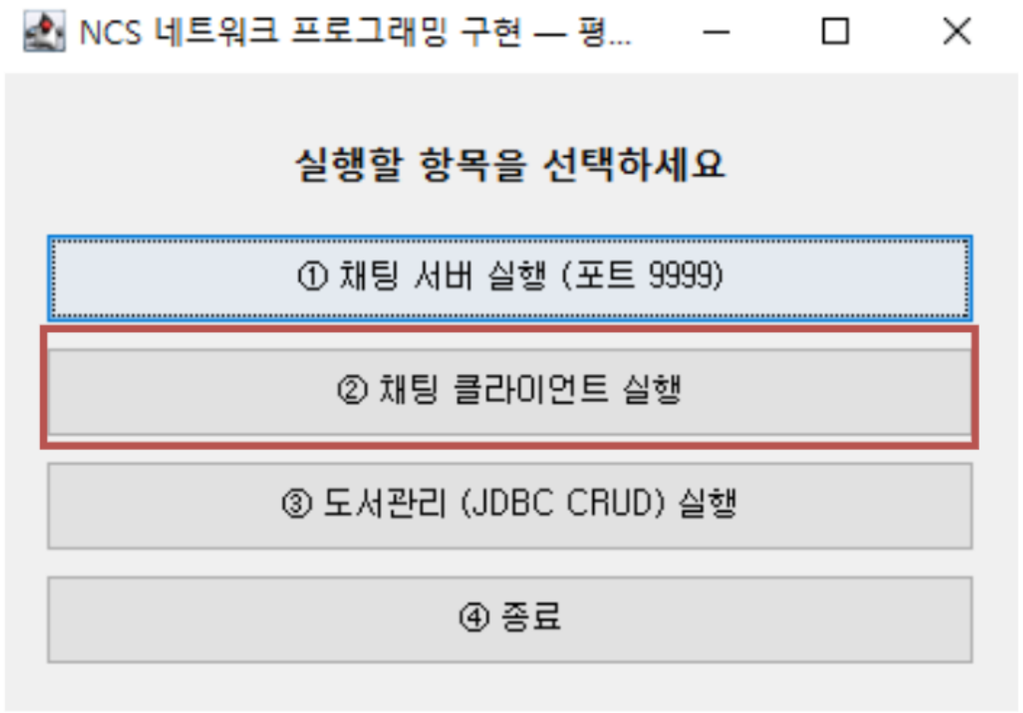
실행 메뉴 중 [채팅 서버 실행 (포트 9999)] 버튼을 클릭하여 소켓 통신을 위한 서버 개방을 준비합니다.

06 | SOCKET TODOS - 작동 결과



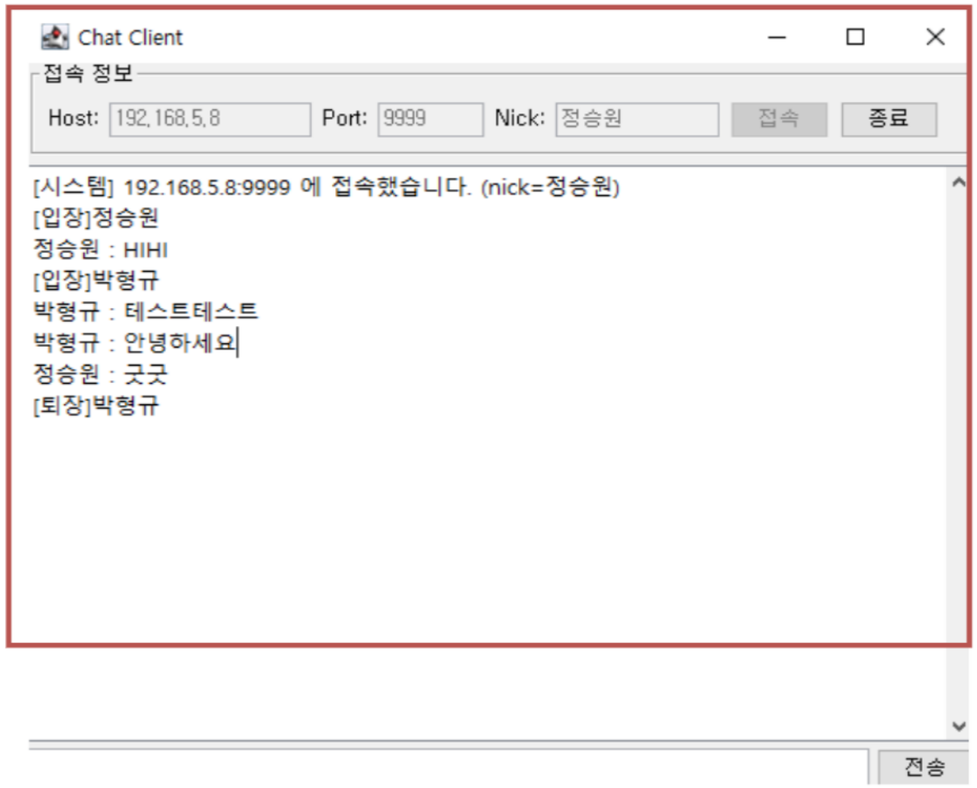
"채팅 서버를 콘솔에서 시작했습니다. (포트 9999)"라는 팝업 알림이 뜨며, 지정된 9999 포트로 서버가 활성화되었음을 보여줍니다.

06 | SOCKET TODOS - 작동 결과



메인 메뉴에서 [채팅 클라이언트 실행] 버튼을 클릭하여 실시간 대화에 참여할 로그인을 준비합니다.

06 SOCKET TODOS - 작동 결과



호스트 IP와 포트 번호, 닉네임을 입력해 서버에 접속한 뒤, 다른 사용자들과 실시간으로 메시지를 주고받는 최종 화면입니다.